

Bàn về ứng dụng của tư tưởng vét cạn đặc biệt

Ge Rong

Trường Trung học số 1 Đường Sơn, Hà Bắc

IOI 2004, đội tuyển tập huấn quốc gia Trung Quốc

Nguồn gốc: Bàn về ứng dụng của tư tưởng vét cạn đặc biệt

IOI China candidate team papers / 2004 / Ge Rong.pdf

Abstract

Bài viết phân tích một số ứng dụng đặc biệt của hai kiểu vét cạn: vét cạn toàn phần và vét cạn từng phần. Qua các ví dụ cụ thể, bài viết cho rằng vét cạn vẫn là một tư tưởng hữu hiệu khi giải bài, và đáng được chú ý đúng mức. Với vét cạn toàn phần, nếu chọn đúng đối tượng cần xét, ta có thể đạt hiệu quả mong muốn. Với vét cạn từng phần, hay phương pháp tham biến, ta có một công cụ mạnh cho các bài toán cực tiểu hóa giá trị lớn nhất hoặc cực đại hóa giá trị nhỏ nhất. Tư tưởng này còn mở rộng phạm vi áp dụng của các thuật toán đồ thị, tham lam và quy hoạch động, trong khi hiệu suất thường vẫn đáp ứng yêu cầu.

Từ khóa: vét cạn; vét cạn toàn phần; vét cạn từng phần; phương pháp tham biến; bài toán cực đại–cực tiểu; ghép cực đại–cực tiểu.

1 Mở đầu

Vét cạn là một trong những tư tưởng quan trọng nhất của tin học. Tốc độ xử lý cao của máy tính làm cho việc xét nhiều khả năng trở nên khả thi. Tuy nhiên, khi lý thuyết đồ thị, số học, quy hoạch động và các thuật toán tìm kiếm ngày càng phát triển, vét cạn dường như ngày càng ít được coi trọng, thậm chí bị đồng nhất với “kém hiệu quả”. Thực ra tư tưởng vét cạn vẫn có nhiều ứng dụng. Bài viết này phân tích vài cách dùng đặc biệt của nó qua một số bài toán tiêu biểu.

2 Tư tưởng vét cạn

Nói ngắn gọn, vét cạn là tư tưởng đưa ra mọi khả năng có liên quan, đôi khi cả một số khả năng sau đó sẽ bị loại, rồi xử lý chúng. Có thể chia thành hai dạng.

Vét cạn toàn phần xét toàn bộ không gian trạng thái đã chọn. Vì dễ sinh ra nhiều trường hợp thừa, nó thường được thay bằng các thuật toán tìm kiếm có cắt tỉa. Dù vậy, ta sẽ thấy vét cạn toàn phần không nhất thiết chậm hơn một phép tìm kiếm có nhiều mẹo cắt tỉa.

Vét cạn từng phần phù hợp với các bài toán có một đại lượng quan trọng chưa biết. Nếu biết đại lượng ấy, ta có thể giải phần còn lại bằng một thuật toán hiệu quả. Khi đó ta chỉ xét đại lượng quan trọng này, rồi dùng thuật toán khác để kiểm tra hoặc xây dựng lời giải.

Khi áp dụng tư tưởng vét cạn, có thể theo bốn bước:

1. Hiểu chính xác đề bài. Đây là bước cần thiết với mọi phương pháp.
2. Xác định rằng bài toán đáng dùng vét cạn. Nếu bài toán có một phương pháp chuyên biệt tốt hơn, nên ưu tiên phương pháp đó. Đồng thời số trạng thái cần xét phải đủ nhỏ để thời gian chạy chấp nhận được.
3. Chọn rõ đối tượng cần vét cạn. Lựa chọn này ảnh hưởng trực tiếp đến độ phức tạp thời gian. Một đối tượng phù hợp có thể biến một lời giải tưởng như thô sơ thành rất hiệu quả.
4. Giải phần còn lại của bài toán dựa trên đối tượng đã chọn.

2.1 Ví dụ 1: Smart Typist

Đề bài. Cho một bàn phím đặc biệt chỉ có sáu phím: Up, Down, Left, Right, Swap0, Swap1. Vùng nhập có sáu vị trí, đánh số từ trái sang phải là 1, 2, ..., 6. Phím Up/Down tăng hoặc giảm chữ số tại vị trí con trỏ một đơn vị, nhưng không vượt khỏi khoảng 0 đến 9. Hai phím Left/Right di chuyển con trỏ sang trái hoặc phải. Phím Swap0 đổi chữ số tại con trỏ với vị trí 1; phím Swap1 đổi chữ số tại con trỏ với vị trí 6. Ban đầu con trỏ ở vị trí 1. Cần biến một mật khẩu sáu chữ số thành một mật khẩu sáu chữ số khác với số lần gõ ít nhất.

Ví dụ, từ trạng thái 123456 muốn đến 633451, có thể làm:

$$123456 \xrightarrow{\text{Swap1}} 623451 \xrightarrow{\text{Right}} 623451 \xrightarrow{\text{Up}} 633451.$$

Phân tích ban đầu. Sự xuất hiện của hai phím hoán đổi làm bài toán có vẻ không còn cấu trúc đơn giản, nên cách tự nhiên là tìm kiếm trạng thái. Nhưng tổng số trạng thái cỡ $6 \cdot 10^6$: sáu vị trí con trỏ và 10^6 chuỗi chữ số. Tìm kiếm trực tiếp tốn nhiều thời gian và bộ nhớ. Có thể dùng hàm đánh giá tốt, cắt tỉa, bảng băm kết hợp A*, hoặc BFS hai chiều, nhưng chương trình sẽ phức tạp và hiệu suất vẫn không thật gọn.

Chọn đối tượng để vét cạn. Khó khăn nằm ở hai phím Swap0 và Swap1. Nếu bỏ hai phím này thì bài toán gần như tầm thường: chỉ cần di chuyển con trỏ và điều chỉnh từng chữ số. Vì vậy ta xét riêng cách dùng hai phím hoán đổi.

Thoạt nhìn, việc này không khả thi: ta không biết thứ tự xen kẽ giữa các phím hoán đổi và các phím tăng giảm, cũng không biết tổng số thao tác. Nhưng có một nhận xét quan trọng: nếu chỉ tăng giảm cùng một chữ số, đặt thao tác Up/Down trước hay sau thao tác hoán đổi là tương đương, miễn là nó vẫn tác động đến chữ số tương ứng. Chẳng hạn từ 133456, ta có thể dùng Up, Right, Swap0 để thu được 323456; cũng có thể dùng Right, Swap0, rồi Up và vẫn thu được kết quả đó.

Do đó ta chia lời giải thành hai pha. Pha thứ nhất chỉ xét các thao tác hoán đổi để tạo ra một hoán vị của sáu chữ số ban đầu. Pha thứ hai không còn dùng hoán đổi, chỉ tính chi phí di chuyển và tăng giảm để đạt mật khẩu đích. Số hoán vị của sáu vị trí chỉ là $6! = 720$, nên có thể dùng vét cạn hoặc BFS trên không gian hoán vị để tính số thao tác hoán đổi nhỏ nhất.

Vẫn còn một chi tiết: trong quá trình tạo một hoán vị, con trỏ có thể chưa đi qua mọi vị trí, nên không phải vị trí nào cũng có thể được điều chỉnh không tốn thêm thao tác Left/Right. Nếu xét trạng thái “vị trí nào đã có thể chỉnh”, có tối đa $2^6 = 64$ khả năng, vẫn chỉ tạo $720 \cdot 64 = 46080$ trạng thái, bằng khoảng 0.77% không gian tìm kiếm trực tiếp.

Phân tích kỹ hơn còn giảm được nữa:

1. Vị trí 1 luôn có thể chỉnh, vì con trỏ xuất phát tại đó.

2. Với các vị trí 2, 3, 4, 5, nếu một vị trí bên phải đã có thể chỉnh thì mọi vị trí bên trái của nó cũng có thể chỉnh trong quá trình đi qua. Vì vậy tập các vị trí chỉnh được trong đoạn này chỉ có năm dạng:

$$\emptyset, \{2\}, \{2, 3\}, \{2, 3, 4\}, \{2, 3, 4, 5\}.$$

3. Vị trí 6 có thể chỉnh hoặc chưa, tùy việc có dùng Swap1 hay chưa.

Tổng cộng chỉ có $2 \cdot 5 = 10$ dạng phụ, tức khoảng 7200 trạng thái. Đây là điểm then chốt: chọn đúng đối tượng vét cạn làm bài toán trở nên nhỏ và dễ cài đặt.

Thuật toán	Độ phức tạp	Cài đặt	Nhận xét
Tìm kiếm trạng thái	rất lớn	khó	chưa tận dụng điều kiện đề
Vét cạn đã chọn	$\mathcal{O}(720 \cdot 10)$	khá nhỏ	khai thác cấu trúc hoán đổi

2.2 Ví dụ 2: Logic Island

Đề bài. Trên Logic Island có ba loại cư dân: thần luôn nói thật, ác quỷ luôn nói dối, và người thường nói thật vào ban ngày nhưng nói dối vào ban đêm. Một nhà xã hội học đến thăm đảo, nhưng không thể phân biệt loại cư dân bằng vẻ ngoài. Trên đảo chỉ có năm cư dân A, B, C, D, E . Mỗi câu nói thuộc một trong ba dạng:

1. I am [not] divine/evil/human/lying.
2. X is [not] divine/evil/human/lying.
3. It is day/night.

Tổng số câu không quá 50. Cần suy ra loại của từng cư dân và thời điểm là ban ngày hay ban đêm, nếu các dữ kiện cho phép.

Xét ví dụ:

A: B is human.
B: A is evil.
A: B is evil.

Nếu suy luận bằng tay, hai câu của A mâu thuẫn nhau, nên A đang nói dối. Khi đó B không thể là người hay ác quỷ trong cấu hình này; B là thần, và vì thần nói thật nên A là ác quỷ. Kết luận là:

A is evil.
B is divine.

Áp dụng vét cạn. Nếu cố mô phỏng kiểu suy luận trên bằng chương trình, các nhánh suy luận khá rối. Nhưng số cư dân chỉ là 5, mỗi người có 3 loại, còn thời gian chỉ có hai khả năng. Tổng số giả thiết là

$$3^5 \cdot 2 = 486.$$

Đây là đối tượng vét cạn tự nhiên. Với mỗi giả thiết, ta kiểm tra từng câu nói có phù hợp với loại của người nói và thời điểm hay không. Những giả thiết phù hợp được giữ lại. Một sự kiện được xem là suy ra chắc chắn nếu nó đúng trong mọi giả thiết còn lại; nếu không có giả thiết nào thì cuộc hội thoại là bất khả.

2.3 Tiểu kết

Hai ví dụ trên cho thấy vết cạn toàn phần có vẻ mù quáng, nhưng người thiết kế thuật toán thì không. Phân tích đề bài đủ sâu và chọn linh hoạt đối tượng cần xét có thể biến một bài toán khó thành một bài toán nhỏ, giảm độ phức tạp cài đặt và vẫn đạt hiệu quả tốt.

3 Vết cạn từng phần: phương pháp tham biến

Các phương pháp như đồ thị, tham lam và quy hoạch động đã rất phát triển, nhưng bản thân chúng vẫn có giới hạn. Nhiều bài toán chỉ vì một điều kiện khó chịu mà ta không áp dụng được thuật toán hiệu quả quen thuộc. Vết cạn từng phần là một cách phá giới hạn ấy.

Phương pháp tham biến hoạt động như sau. Trước hết ta xét một kết quả quan trọng của bài toán như một tham biến. Sau đó dùng tham lam, quy hoạch động, đồ thị hoặc một công cụ khác để kiểm tra tham biến đó có khả thi hay không, đồng thời suy ra những thành phần còn lại của lời giải.

không gian lời giải đầy đủ, quá lớn để xét trực tiếp

↓

chọn một kết quả quan trọng làm tham biến

↓

xét tham biến và kiểm tra khả thi bằng thuật toán hiệu quả

↓

xây dựng các kết quả còn lại

Vì chỉ xét tham biến quan trọng, còn phần khác được xử lý bằng thuật toán hiệu quả, độ phức tạp thường là đa thức. Đôi khi vẫn cần tận dụng thêm tính chất riêng của đề để tối ưu.

Tính chất quan trọng nhất thường là **đơn điệu**. Nếu tham biến x_1 khả thi thì mọi $x \geq x_1$ cũng khả thi, và bài toán yêu cầu giá trị nhỏ nhất của tham biến, ta có hai lựa chọn:

1. xét tuyến tính và dừng ở nghiệm khả thi đầu tiên;
2. xét bằng chia đôi.

Chia đôi bảo đảm độ phức tạp thấp hơn, nhưng cài đặt phức tạp hơn một chút. Với miền giá trị nhỏ, hoặc khi nghiệm thường nằm gần biên, cách tuyến tính có thể tiết kiệm thời gian lập trình và giảm lỗi. Dù chia đôi không duyệt từng giá trị một, tư tưởng của nó vẫn thống nhất với vết cạn từng phần.

Nhìn chung, các bài toán cực tiểu hóa giá trị lớn nhất hoặc cực đại hóa giá trị nhỏ nhất rất hợp với phương pháp này. Chẳng hạn bài toán định nghĩa một trọng số và yêu cầu tạo một phân hoạch sao cho trọng số lớn nhất của mỗi phần được nhỏ nhất. Ta thường chuyển thành: đã biết một ngưỡng x , có tồn tại một phân hoạch thỏa yêu cầu hay không? Bài toán kiểm tra này là điểm mấu chốt: nó phải đúng, được nghiệm khi cần, và đủ nhanh vì có thể bị gọi nhiều lần.

3.1 Ví dụ 3: Strawberry

Đề bài. Các tinh linh trong rừng muốn chia một ruộng dâu thành k mảnh và trồng vào k khoảng đất trống. Mỗi mảnh phải liên thông theo các tua nối giữa các cây dâu. Nếu một mảnh có tổng khối lượng quá nhỏ thì các cây dâu sẽ cô đơn, nên các tinh linh muốn mảnh nhẹ nhất càng nặng càng tốt.

Ký hiệu sum_i là tổng khối lượng dâu trong mảnh thứ i , với $1 \leq i \leq k$. Đặt

$$x = \min\{\text{sum}_i \mid 1 \leq i \leq k\}.$$

Cần chia ruộng dâu thành k mảnh liên thông sao cho x lớn nhất, rồi in phương án và giá trị x .

Bản gốc là một bài nộp đáp án. Nhiều dữ liệu của bài có dạng cây; ở đây ta xét trường hợp cấu trúc là cây và cần nghiệm tối ưu.

Phân tích ban đầu. Khi gặp bài toán tối ưu trên cây, ta dễ nghĩ đến quy hoạch động. Gọi $D(v, k)$ là giá trị tốt nhất khi chia cây con gốc v thành k phần. Nếu v có các con $c_1, \dots, c_t$, một công thức tự nhiên là xét mọi cách tách

$$k = a_1 + a_2 + \dots + a_t$$

rồi kết hợp các giá trị

$$D(c_1, a_1), D(c_2, a_2), \dots, D(c_t, a_t),$$

kèm điều chỉnh do gán đỉnh v vào một phần nào đó. Vấn đề nằm ở phần điều chỉnh này: để biết sau khi thêm v thì mảnh nhỏ nhất thay đổi thế nào, ta không chỉ cần giá trị nhỏ nhất mà còn có thể cần giá trị nhỏ thứ hai, rồi thứ ba, và cuối cùng gần như phải lưu mọi khả năng. Công thức tuy có vẻ đúng nhưng không khả thi về thời gian và bộ nhớ.

Chọn tham biến. Giả sử đã biết một giá trị x . Ta hỏi: có thể chia cây thành ít nhất k mảnh liên thông, mỗi mảnh có tổng khối lượng ít nhất x , hay không? Nếu trả lời được câu hỏi này, ta có thể xét các giá trị x và tìm giá trị lớn nhất khả thi.

Câu hỏi kiểm tra thực ra không khó. Duyệt cây từ dưới lên. Với mỗi đỉnh v :

- $T(v)$ là số mảnh đã tách được trong cây con của v , mỗi mảnh có khối lượng ít nhất x ;
- $L(v)$ là khối lượng còn dư lớn nhất chưa tách thành mảnh sau khi đã đạt $T(v)$.

Gọi $w(v)$ là khối lượng của v , và các con của v là $c_1, \dots, c_t$. Đặt

$$R = w(v) + \sum_{i=1}^t L(c_i).$$

Nếu $R \geq x$, ta tách thêm một mảnh chứa v và các phần dư từ con:

$$T(v) = 1 + \sum_{i=1}^t T(c_i), \quad L(v) = 0.$$

Nếu $R < x$, ta chưa tách được mảnh mới:

$$T(v) = \sum_{i=1}^t T(c_i), \quad L(v) = R.$$

Đây có thể xem là quy hoạch động, cũng có thể xem là tham lam: xử lý từ lá lên gốc, gặp nơi đủ khối lượng thì cắt ngay. Quy hoạch động cho chứng minh đúng khá rõ; kết quả tham lam trùng với quy hoạch động nên thuật toán tham lam cũng đúng.

Vì tính khả thi đơn điệu theo x : nếu chia được với ngưỡng x thì cũng chia được với mọi ngưỡng nhỏ hơn, ta dùng chia đôi trên x . Mỗi lần kiểm tra mất $\mathcal{O}(n)$ trên cây, nên tổng thời gian là $\mathcal{O}(n \log W)$, với W là tổng khối lượng.

3.2 Ghép cực đại–cực tiểu

Vết cạn từng phần cũng xuất hiện trong các thuật toán kinh điển. Một ví dụ quan trọng là ghép hoàn hảo trong đồ thị hai phía có trọng số, tối ưu theo ngưỡng biên.

Bài toán. Cho một đồ thị hai phía có trọng số. Cần tìm một ghép hoàn hảo sao cho trọng số lớn nhất trong các cạnh được chọn là nhỏ nhất. Nếu đặt

$$x = \max\{w(e) \mid e \text{ là cạnh được chọn}\},$$

ta cần cực tiểu hóa x .

Phân tích. Bài toán không chuyển trực tiếp thành ghép cực đại hay ghép trọng số cực đại quen thuộc. Mô hình luồng mạng thông thường cũng không khớp ngay, vì thông tin cần tối ưu là một giá trị biên, không phải tổng trọng số cộng được.

Nếu biết một ngưỡng x , bài toán kiểm tra trở nên đơn giản: chỉ giữ những cạnh có trọng số không vượt quá x . Ta thu được một đồ thị hai phía không trọng số. Nếu trong đồ thị này tồn tại ghép hoàn hảo thì có một nghiệm có giá trị biên không vượt quá x ; ngược lại, ngưỡng x không đủ.

Về cách xét x , miền trọng số có thể rất lớn, nhưng các giá trị có hiệu lực chỉ là trọng số của các cạnh, nhiều nhất n^2 giá trị với đồ thị đầy đủ $n \times n$. Có thể sắp xếp các trọng số rồi xét tuyến tính; để ổn định hơn về thời gian, thường dùng chia đôi trên danh sách trọng số đã sắp xếp.

Cùng một khuôn mẫu còn xử lý được biến thể đối xứng: tìm ghép hoàn hảo sao cho trọng số nhỏ nhất trong các cạnh được chọn là lớn nhất. Khi đó kiểm tra ngưỡng x bằng cách giữ các cạnh có trọng số ít nhất x , rồi tìm ghép hoàn hảo.

4 Kết luận

Bài viết dùng vết cạn toàn phần và vết cạn từng phần để xử lý một số bài toán. Từ các ví dụ có thể thấy:

$$\text{vết cạn} \neq \text{kém hiệu quả}.$$

Với vết cạn toàn phần, việc chọn đúng đối tượng quyết định hiệu quả. Với vết cạn từng phần, phương pháp tham biến là công cụ mạnh cho các bài toán cực trị dạng biên, đồng thời mở rộng phạm vi áp dụng của đồ thị, tham lam và quy hoạch động.

Tiêu chí	Vết cạn toàn phần	Vết cạn từng phần
Thời gian	phụ thuộc mạnh vào đối tượng	thường khá tốt
Bộ nhớ	thường tốt	phụ thuộc thuật toán kiểm tra
Cài đặt	tương đối thấp	phụ thuộc thuật toán kiểm tra
Điểm khó	chọn đối tượng	chọn tham biến
Phạm vi	khi thiếu phương pháp chuyên biệt	bài toán cực trị dạng biên

Tóm lại, cả vết cạn toàn phần và vết cạn từng phần đều là tư tưởng hữu hiệu trong giải bài. Ta nên lựa chọn theo tính chất cụ thể của đề, thay vì mặc định loại bỏ vì cho rằng mọi phương pháp vết cạn đều chậm.

Tài liệu tham khảo

1. Yu Wei, báo cáo lời giải NOI 2001.
2. Wu Wenhua và Wang Jiande, *Thuật toán và lập trình đồ thị*, Nhà xuất bản Đại học Thanh Hoa.

A Phụ lục: đề và giả mã tham khảo

A.1 Smart Typist

Mỗi mật khẩu có độ dài 6, con trỏ ban đầu ở vị trí 1. Các phím Swap0, Swap1, Up, Down, Left, Right hoạt động như đã mô tả ở phần chính. Ví dụ đầu vào và đầu ra:

Input

123456 654321

Output

11

Một chuỗi thao tác tối ưu là:

123456

Swap1 -> 623451

Right -> 623451

Swap0 -> 263451

Down -> 253451

Right -> 253451

Up -> 254451

Right -> 254451

Down -> 254351

Right -> 254351

Up -> 254361

Swap0 -> 654321

Giả mã dưới đây ghi lại ý tưởng chính của chương trình trong bản gốc: xét các cách dùng hai phím hoán đổi, rồi cộng chi phí chỉnh chữ số và di chuyển.

```
best = infinity
for each reachable permutation p created by Swap0/Swap1:
    for each accessibility pattern a of cursor positions:
        cost = swap_cost[p, a]
        current = apply_permutation(start, p)
        cost += digit_change_cost(current, target)
        cost += minimum_cursor_movement_needed(a, current, target)
        best = min(best, cost)
print best
```

A.2 Logic Island

Với mỗi cuộc hội thoại, xét 486 giả thiết về loại của năm cư dân và thời điểm. Sau đó hợp các giả thiết hợp lệ để tìm sự kiện chắc chắn.

```

possible = empty list
for mask from 0 to 485:
    decode mask into type[A..E] and day_or_night
    derive lying[A..E] from type and time
    ok = true
    for each statement s:
        value = truth_value_of_statement(s, type, time, lying)
        if value is not compatible with lying[speaker(s)]:
            ok = false
            break
    if ok:
        add this assignment to possible

if possible is empty:
    print "This is impossible."
else:
    print every resident type and time value common to all assignments

```

A.3 Strawberry trên cây

Với một ngưỡng x , thủ tục kiểm tra sau trả về liệu có thể tách được ít nhất k mảnh liên thông có tổng khối lượng không nhỏ hơn x hay không.

```

check(x):
    pieces = 0
    dfs(v, parent):
        rest = weight[v]
        for each child u of v:
            dfs(u, v)
            rest += leftover[u]
        if rest >= x:
            pieces += 1
            leftover[v] = 0
        else:
            leftover[v] = rest
    dfs(root, none)
    return pieces >= k

```

answer = largest x such that check(x) is true

Nếu cần in phương án, khi một đỉnh v tạo mảnh mới trong check, ta thu các đỉnh thuộc phần dư liên thông dưới v ; các đỉnh còn lại có thể được ghép vào một mảnh hợp lệ cuối cùng theo cấu trúc cây.

A.4 Ghép theo ngưỡng biên

Phiên bản cực tiểu hóa cạnh lớn nhất:

```

sort all distinct edge weights increasingly into values
binary search the smallest index i:

```



```
x = values[i]
build bipartite graph H with edges e where weight[e] <= x
feasible = H has a perfect matching
print values[i]
```

Phiên bản cực đại hóa cạnh nhỏ nhất đổi điều kiện thành $\text{weight}[e] \geq x$ và tìm ngưỡng lớn nhất còn khả thi.